ENTERPRISE ARCHITECTURAL SPECIFICATION

Retail Customer Intelligence Platform

Comprehensive Backend Systems Architecture, Big Data Pipeline, Machine Learning Infrastructure, and Serving Mechanics

Author / Team: Data Engineering & ML Architecture

Platform: DashFlow / INSIGHT.arc v2.4

Repository: retail-customer-intelligence

Target: Production Dual-Execution (Render + Vercel)

Executive Architecture Summary

The Retail Customer Intelligence Platform is an end-to-end Big Data & Machine Learning system designed to ingest, process, clean, and enrich high-volume retail transactions (1.06M rows) and live commercial product catalogs. It computes behavioral RFM cohorts, predicts 90-day Customer Lifetime Value (CLV), forecasts churn risk, and serves sub-millisecond customer profiles to enterprise executive dashboards.

1. High-Level Architectural Topology

The backend is structured according to a modern decoupled Lambda/Kappa Big Data architecture, separating the batch analytical compute layer (PySpark/Hive) from the low-latency Key-Value serving layer (HBase/SQLite) and the application API gateway (Flask/Gunicorn):

```
+-------------------------------------------------------------------------------------------------
-+ | DATA INGESTION LAYER | | [Real Shopify Crawlers: Allbirds, ColourPop, Bulletproof] [UCI Online
Retail II: 1.06M Rows] | +-------------------------------------------------------------------------
-------------------------+ | v +-------------------------------------------------------------------
--------------------------------+ | HDFS IMMUTABLE RAW LANDING ZONE | |
/retail/raw/products/<YYYY-MM-DD>/products.csv /retail/raw/transactions/transactions.csv| +--------
-----------------------------------------------------------------------------------------------+ | v +-
-----------------------------------------------------------------------------------------------------+
| HIVE / PYSPARK DATA WAREHOUSE LAYER | | 1. Data Sanitization & Cancellation Filtering ('C%'
invoices dropped, positive quantity & price) | | 2. RFM Aggregation Engine: Computes Recency,
Frequency, Spend & Tenure across 5,878 customers | | 3. Catalog Benchmark Enrichment: 148 product
categories, average price & rating distributions | | OUTPUT: Optimized Columnar Parquet
(data/warehouse/clean_transactions, rfm_aggregates, enrich) | +-----------------------------------
-------------------------------------------------------------+ | v +-----------------------------
-------------------------------------------------------------------+ | SPARK ML INTELLIGENCE &
MODEL ENGINE | | - K-Means Clustering: 4 Behavioral Cohorts (Champions, Loyal, Potential, At-Risk)
| | - Gradient Boosting Regressor: 90-Day CLV Prediction (99% Cap winsorization, R²=0.5238,
MAE=$321) | | - Random Forest Classifier: Churn Prediction (ROC-AUC = 0.8056, F1 = 0.7876) | |
OUTPUT: Serialized Scikit-Learn / Joblib Artifacts (models/*.joblib) | +-------------------------
-------------------------------------------------------------------------+ | v +-------------------
-------------------------------------------------------------------------------+ | HBASE / NOSQL
LOW-LATENCY SERVING STORE | | Column Families: cf_rfm (Metrics), cf_pred (ML Scores), cf_hist
(Orders), cf_bench (Catalog) | | Storage Engine: Embedded NoSQL KV Store (SQLite / HappyBase) |
Latency: 0.84 ms (p99) | +-------------------------------------------------------------------------
-------------------------+ | v +-------------------------------------------------------------------
--------------------------------+ | FLASK MASTER APPLICATION & API ROUTER | | REST Endpoints:
/api/customer/<id>, /api/predict, /api/analytics, /api/health, /api/v1/* | | Enterprise
Persistence: web/db.py (Users, Saved Scenarios, Dataset Imports, Customer Notes) | +---------------
-------------------------------------------------------------------------+ | v +--------
-----------------------------------------------------------------------------------+ |
PRESENTATION & CLIENT DELIVERY | | Vercel Edge CDN Frontend (React + TypeScript) <---> Render WSGI
```

```
Backend Server (Gunicorn) | +-------------------------------------------------------------------------
----------------------------+
```

2. Layer-by-Layer Working Mechanics

Layer 1: Web Scraper & Catalog Crawler (`scraper/`)

Rather than relying on synthetic or toy product data, the platform crawls live commercial e-commerce stores running on public Shopify architectures (Allbirds, ColourPop, Bulletproof, UNTUCKit, Taylor Stitch).

- **Mechanism:** Sends paginated requests to `/products.json?limit=250&page={n}` with user-agent rotation and rate-limiting delays (`0.4s`).
- **Field Normalization:** Extracts product SKUs, titles, normalized categories (e.g., *Shoes, Cosmetics, Coffee, Outerwear*), primary variant prices, compare-at MSRP, vendor names, stock availability flags, and review sentiments.
- **Output Artifacts:** Saves timestamped raw partitions at `data/raw/products/<YYYY-MM-DD>/products.csv` and `products.json` (over 2,038 authentic live items).

Layer 2: HDFS Raw Zone & Hive Analytical Warehouse (`hdfs/` , `hive/`)

The ingestion and warehouse layers ensure raw data integrity and transform transactional event streams into analytical data marts:

Raw Landing Zone (HDFS)

Stores unmodified transactions and crawled product files in HDFS paths: `/retail/raw/transactions/` and `/retail/raw/products/` . Guarantees immutable auditability and historical replayability.

Hive / PySpark ETL Engine

Executed via `hive/run_hive_pipeline.py` . Performs distributed data cleaning on 1,067,371 rows: filters cancelled invoices (prefix `'C'`), null customer IDs, and non-positive prices.

The RFM Transformation Algorithm:

1. **Order Aggregation:** Groups transactions by `(customer_id, invoice)` to calculate per-order totals: $$ ext{order_amount} = \sum (ext{quantity} imes ext{price})$$
2. **Customer Metrics:** Groups orders by `customer_id` to determine:
   - **Recency (R):** Days between the global dataset snapshot date ($T_{ ext{snapshot}}$) and the customer's last purchase date.
   - **Frequency (F):** Total count of distinct confirmed invoices.
   - **Monetary (M):** Cumulative spend across all confirmed purchases.
   - **Tenure (T):** Total days elapsed since the customer's very first recorded transaction.
3. **Columnar Parquet Warehouse:** Writes results into Snappy-compressed Parquet files (`data/warehouse/rfm_aggregates/` and `clean_transactions/`) offering 8.4x storage reduction and high-speed columnar scans.

Layer 3: Machine Learning Engine & Feature Store (`spark/` , `models/`)

The machine learning pipeline (`spark/train_models.py`) trains three predictive and prescriptive models:

Model	Algorithm & Hyperparameters	Features & Transformations	Evaluation Metrics
RFM Customer Segmentation	K-Means Clustering ($k=4$, `k-means++`, random_state=42)	$\log(1 + R), \log(1 + F), \log(1 + M)$ normalized via `StandardScaler` to eliminate power-law skew.	Distinct separation into 4 cohorts: *Champions, Loyal Customers, Potential Loyalists, At-Risk*.
Customer Lifetime Value (CLV)	Gradient Boosting Regressor (`n_estimators=100`, `learning_rate=0.05`, `max_depth=4`)	$R, F, M, T, ext{AOV}$, plus real catalog benchmark price and rating. Winsorized at 99th percentile ($6,245.56 cap).	**$R^2 = 0.5238$** MAE = **$321.78** RMSE = **$606.12**
Churn Prediction Classifier	Random Forest Classifier (`n_estimators=100`, `max_depth=6`, `class_weight='balanced'`)	Full behavioral profile. Ground truth label: customer has 0 orders in the final 90-day holdout observation window.	**ROC-AUC = 0.8056** Accuracy = **75.09%** F1-Score = **0.7876**

3. Low-Latency Serving Layer Mechanics (`hbase/`)

Analytical warehouses (Hive/Spark) are designed for heavy batch jobs, not sub-second UI interactions. The platform solves the *"Dual-Speed Big Data"* problem using an HBase/NoSQL columnar key-value serving layer (`hbase/serving.py`).

Column Family Schema Specification

Column Family	Qualifier / Key	Data Type	Description
`cf_rfm`	`recency` , `frequency` , `monetary` , `tenure` , `aov`	Integer / Float	Aggregated historical behavioral metrics.
`cf_pred`	`segment` , `predicted_clv` , `churn_probability`	String / Float	Pre-computed model inference scores.
`cf_hist`	`recent_orders` , `total_orders`	JSON String	Recent transaction records and item counts.
`cf_bench`	`benchmark_price` , `benchmark_rating`	Float	Real commercial store category price averages.

Dual-Driver Architecture:

- **HBase Driver (Production Cluster):** Connects via Thrift/HappyBase to Apache HBase RegionServers for distributed low-latency row key queries.
- **Embedded SQLite KV Engine (Local / Standalone):** Provides an embedded, zero-dependency SQLite key-value store (`data/serving/customer_profile.db`) with 5,942 pre-indexed customer rows. Delivers a guaranteed **0.84 ms p99 latency** without requiring a full distributed HBase cluster on developer machines.

4. Enterprise Database Layer & Persistence (`web/db.py`)

In addition to the read-only customer analytics serving store, `web/db.py` implements a relational schema for operational state, user authentication, and scenario auditability:

- `users` : Manages credentials (salted PBKDF2/SHA256 password hashing), roles (`admin` , `analyst`), and session records.
- `saved_scenarios` : Persists What-If simulations created by business analysts, including adjusted parameters (R, F, M, Tenure) and model output scores.
- `dataset_imports` : Tracks file uploads, record counts, and pipeline execution logs.
- `customer_notes` : Stores account-level CRM notes, assigned tasks, and outreach history.

5. Flask Application Server & REST API (`web/app.py` , `web/api_v1.py`)

The application server runs on Flask and Gunicorn, exposing clean RESTful JSON endpoints:

Endpoint	Method	Function & Mechanics	Latency
`/api/customer/<id>`	`GET`	Point-lookup in NoSQL serving store. Returns full profile, CLV, churn, RFM, and history.	`< 1 ms`
`/api/predict`	`POST`	Real-time ML inference for What-If scenario simulator. Scores arbitrary parameters instantly.	`< 8 ms`
`/api/analytics/revenue-orders`	`GET`	Streams historical monthly and daily revenue and order volume trajectories for charts.	`< 5 ms`
`/api/analytics/cohorts`	`GET`	Returns customer counts, percentages, and monetary volume across the 4 K-Means segments.	`< 4 ms`
`/api/health`	`GET`	System heartbeat: reports p99 latency, serving records count (5,942), and model engines.	`< 1 ms`
`/api/customer/<id>/export/csv`	`GET`	RFC 4180 compliant CSV export with formula injection sanitization (strips `=, +, -, @`).	`< 10 ms`

6. Production Deployment & Verification Architecture

Backend Service (Render WSGI)

- Configured in `render.yaml` & `Procfile`.
- Command: `gunicorn web.app:app --bind 0.0.0.0:$PORT --workers 4`
- Hosts REST APIs, ML model inference, and SQLite/PostgreSQL persistence.

Edge Frontend (Vercel CDN)

- Configured in `vercel.json` & `frontend/vercel.json`.
- Hosts React TypeScript application and static Web3-Verse assets.
- Proxy rewrites route `/api/*` transparently to the Render backend.

Comprehensive Automated Testing Suite

The backend is accompanied by automated test suites to ensure 100% data integrity and continuous delivery reliability:

- `tests/test_platform.py` (**18/18 Tests Passing**): Verifies data sanitization rules, customer profile accuracy, RFM feature scaling, model prediction bounds, and serving store latency.
- `tests/test_api_v1.py` (**11/11 Tests Passing**): Verifies enterprise authentication, token handling, What-If scenario persistence, and dataset import APIs.
- `tests/test_frontend_integrity.py` (**3/3 Tests Passing**): Enforces strict code synchronization between `web/templates/` and `frontend/` to prevent configuration drift.

Summary Checklist: How to Run the Pipeline from Scratch

1. **Scrape:** `python3 scraper/run_scraper.py` — Crawls real Shopify stores into `data/raw/products/`.
2. **Clean & Warehouse:** `python3 hive/run_hive_pipeline.py` — Builds clean Parquet tables and category benchmarks.
3. **Train Models:** `python3 spark/train_models.py` — Fits CLV, Churn, and K-Means models and outputs predictions.
4. **Populate Serving:** `python3 scripts/populate_serving_store.py` — Loads 5,942 profiles into KV serving store.
5. **Start Backend:** `python3 web/app.py` — Launches live Flask server on port 8501.